QUESTION 1

What happens behind the scenes when I log in to WhatsApp?

When I open WhatsApp and tap to log in, several things happen behind the scenes:

Frontend: The frontend is what I see and interact with on my phone, such as the login screen and buttons. When I tap to log in, the frontend sends my login information to the backend.

Backend: The backend receives the login request and checks whether my information is valid. It handles the login process and determines whether I should be allowed to access my account.

Database: The backend may communicate with databases to retrieve the information needed to verify my account and load relevant account data.

Authentication: If my information is verified successfully, the backend confirms that I can access my account. If something is wrong, it returns an error message.

Response to the frontend: The backend sends a response to my phone indicating whether the login was successful.

Home screen: If the login succeeds, the frontend displays my WhatsApp home screen, where I can see my chats and start messaging.

1 B

My own analogy for the backend I would compare the backend to a restaurant kitchen. When I visit a restaurant, I look at the menu and place my order with the waiter. The waiter represents the frontend because they interact directly with me. The kitchen represents the backend because it receives the order, processes it, and prepares the food. The pantry represents the database because it stores the ingredients needed to prepare the meals.

QUESTION 2

Why is it wrong to say Node.js is a programming language?

It is wrong to say Node.js is a programming language because JavaScript is the programming language, while Node.js is a runtime environment that allows JavaScript to run outside a web browser.

A runtime is an environment that provides the tools and features needed to execute a program. It allows the program to run and interact with the computer and other systems.

Google's V8 engine is an important part of Node.js. It executes JavaScript code by compiling it into machine code that the computer can run. Node.js also provides additional features, such as working with files, handling network requests, and communicating with databases. For example, JavaScript can run in a browser to make a website interactive. With Node.js, I can use JavaScript to build a backend server that receives requests and sends responses to users.

2B

What problem did Ryan Dahl solve by taking JavaScript out of the browser?

Before Node.js was introduced in 2009, JavaScript was mainly used inside web browsers to make websites interactive. Developers generally had to use other programming languages to build server-side applications.

Ryan Dahl introduced Node.js to allow JavaScript to run outside the browser, making it possible to use the same language for both frontend and backend development.

Node.js also uses an event-driven, non-blocking I/O model, which allows it to handle many connections efficiently without waiting for each operation, such as reading a file or receiving data from a network, to finish before continuing other work. As a result, JavaScript developers could use Node.js to:

1. Build web servers and backend APIs.
2. Handle requests from multiple users.
3. Read and write files on a server.
4. Communicate with databases.
5. Build real-time applications, such as chat applications.

QUESTION 3

How can one worker handle many customers efficiently?

Imagine a restaurant where one waiter serves many customers. Instead of standing beside each customer until their food is ready, the waiter takes their orders, sends them to the kitchen, and moves on to serve other customers while the food is being prepared.

Here is how the scenario relates to Node.js:

Event Queue: This is like a list of customer orders waiting for the waiter to handle them. In Node.js, incoming events and callbacks are queued for processing.

Event Loop: This is like the waiter checking which orders or tasks need attention next. The event loop helps Node.js process tasks and handle completed operations without blocking while waiting for other tasks.

Thread Pool: This is like the kitchen staff preparing food while the waiter continues serving other customers. In Node.js, the thread pool handles certain time-consuming tasks, such as some file system operations, in the background.

Callback: This is like the kitchen notifying the waiter when an order is ready. In Node.js, a callback is a function that can be called when an operation finishes, allowing the program to handle the result.

This approach allows Node.js to handle many requests efficiently because it does not need to create a separate thread for every request. While some operations are waiting to finish, the event loop can continue handling other work.

3B

When would this approach fail badly ?

Imagine that one customer asks the waiter to complete a very complicated task that takes all their attention for a long time. While the waiter is busy with that one task, other customers have to wait because the waiter cannot serve them. The same thing can happen in Node.js if the server performs a long-running, CPU-intensive operation on its main thread, such as calculating millions of complex mathematical operations in a loop.

For example:

```
app.get("/calculate", (req, res) => {
  let total = 0;

  for (let i = 0; i < 10_000_000_000; i++) {
    total += i;
  }

  res.send("Calculation complete");
});
```

The loop occupies the main thread until the calculation finishes. During that time, the event loop cannot process other incoming requests normally. As a result: Other users may experience very slow responses. Requests may remain waiting for a long time. The server may appear unresponsive.

QUESTION 4

The student gets an error because document.getElementById("name") is a browser feature. The browser provides the document object to allow JavaScript to access and manipulate HTML elements on a webpage. Node.js runs JavaScript outside the browser, so it does not have the browser's document or window objects by default. The global object in Node.js is called global (although globalThis is the standard cross-environment way to access the global object).

For example:

```
// Browser JavaScript
document.getElementById("name");

// Node.js
console.log("Hello from Node.js!");

console.log(global);
```

QUESTION 5

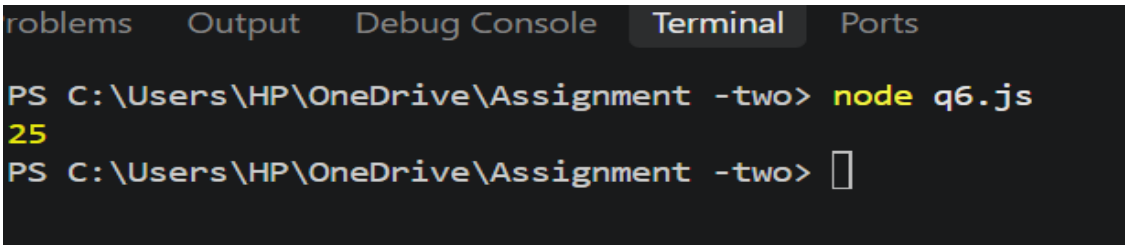
Why is it dangerous to rely only on client-side validation?

It is dangerous because client-side JavaScript runs in the user's browser, and users can bypass or modify it. Even if the signup form checks that an email is valid and a password meets the required rules, someone could still send invalid data directly to the backend without using the form. For example, an attacker could use a tool like Postman or write a script to send a request directly to the server with an invalid email or a password that does not meet the requirements. They could also disable JavaScript or modify the browser's code to bypass the checks. If the backend does not validate the data again, it might accept invalid or malicious input, potentially causing security problems, corrupted data, or

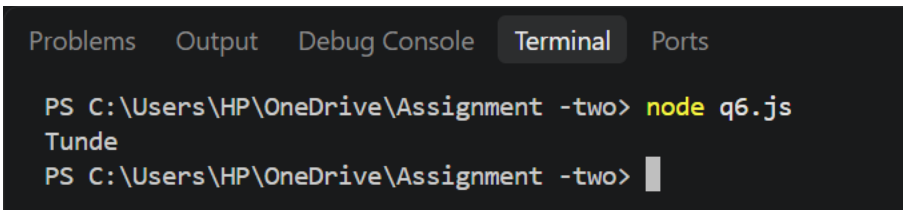
unexpected application behavior. Therefore, the backend must always validate incoming data, even when the frontend already performs validation. Client-side validation improves the user experience by identifying mistakes quickly, while server-side validation protects the application and its database.

QUESTION 6

LINE 1: the variable name is not descriptive. So instead of having 'a' we should have age so the correct code is let age = 25

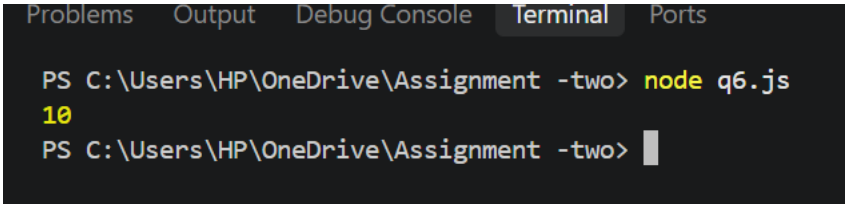


LINE 2: My-Name is wrong because variable names cannot contain a dash (-). I changed it to myName. Which is the camelCase. So, the correct code is:



let myName = "Tunde";

LINE 3: the variable name let is a Javascript keyword used to declare variables and cannot be used as a variable name. The correct code is: const score = 10;



LINE 4: the variable name starts with a number JavaScript variable names cannot begin with a number they can begin with a letter, an underscore (_), or a dollar sign ($). The correct code is: let secondPlace = "life";

```
PS C:\Users\HP\OneDrive\Assignment -two> node q6.js
ife
PS C:\Users\HP\OneDrive\Assignment -two>
```

LINE 5: there is no error here

LINE 6: JavaScript does not allow reassignment of a const variable. This causes a TypeError when the assignment is executed. The best one to use is :

let pi = 3.14;

pi = 3.14159;

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\HP\OneDrive\Assignment -two> node q6.js
3.14159
PS C:\Users\HP\OneDrive\Assignment -two>
```

LINE 7: The variable name does not follow the preferred camelCase style for the intended meaning. Which is let userAge = 30;

LINE 8: Both lines are in the same scope, declaring userAge twice with let causes a Syntax Error. This is the right one to use:

userAge = 30;

userAge = 31;

```
roblems  Output  Debug Console  Terminal  Ports

PS C:\Users\HP\OneDrive\Assignment -two> node q6.js
31
PS C:\Users\HP\OneDrive\Assignment -two>
```

I would use const when I don't plan to reassign a variable after declaring it. For example, if I store someone's name, I can use const because I don't need to assign a different value to that variable.

```
const userName = "Tunde";
```

I would use let when I know the value of a variable needs to change during the program. For example, a user's age or score might change.

```
let score = 10;

score = 20;
```

2. **Why do many developers start with const and only change to let when necessary?**

Many developers start with const because it makes their code safer and easier to understand. It prevents a variable from being reassigned accidentally. If they later discover that the value needs to change, they can switch to let. This approach helps avoid unexpected changes and makes it easier to understand which variables are supposed to stay the same and which ones can change.

3. Why should we avoid using var?

We avoid var because it has some confusing behaviors. Unlike let and const, var is function scoped rather than block scoped, meaning it can be accessed outside a block where it was declared.

For example:

```
if (true) {
   var name = "Tunde";
}


console.log(name); // "Tunde"
```

With let, the variable stays inside the block. Also, var allows you to redeclare the same variable in the same scope, which can lead to mistakes.

QUESTION 7

1. Creating the student object works

```
const student = { name: "Chidi", score: 50 };
```

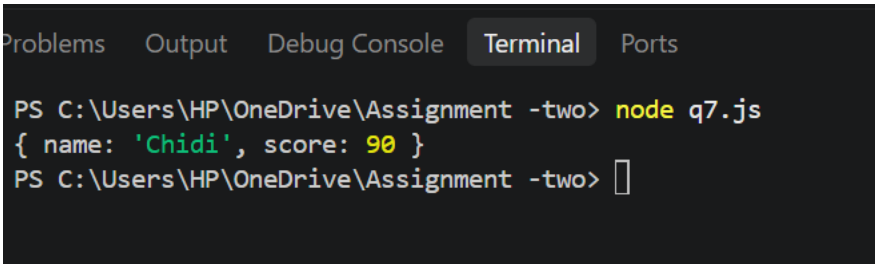
This creates an object with a name and a score. The const keyword prevents us from reassigning the student variable to a different object.

2. Changing the student's score works

```
student.score = 90;

console.log(student);
```

Output:



This works because const does not prevent us from changing the properties inside an object. It only prevents us from reassigning the variable to a different value.

3. Creating the grades array Works

```
const grades = [1, 2, 3];
```

This creates an array containing three numbers.

4. Adding a grade Works

```
grades.push(4);

console.log(grades);
```

Output:

```
roblems  Output  Debug Console  Terminal  Ports

PS C:\Users\HP\OneDrive\Assignment -two> node q7.js
[ 1, 2, 3, 4 ]
PS C:\Users\HP\OneDrive\Assignment -two>
```

This works because push() modifies the existing array instead of assigning a new array to the variable.

5. Reassigning the student variable throws an error

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\HP\OneDrive\Assignment -two> node q7.js
C:\Users\HP\OneDrive\Assignment -two\q7.js:12
student = { name: "Bola", score: 70 };
        ^

TypeError: Assignment to constant variable.
    at Object.<anonymous> (C:\Users\HP\OneDrive\Assignment -two\q7.j
    at Module._compile (node:internal/modules/cjs/loader:1929:14)
    at Object..js (node:internal/modules/cjs/loader:2060:10)
```

This throws a TypeError because student was declared using const. we cannot reassign it to a completely different object.

7B

Using Object.freeze() to make the student object unchangeable

JavaScript provides a method called Object.freeze() that prevents an object's existing properties from being changed, prevents new properties from being added, and prevents existing properties from being removed.

Here is an example:

```
"use strict";


const student = {
  name: "Chidi",
```

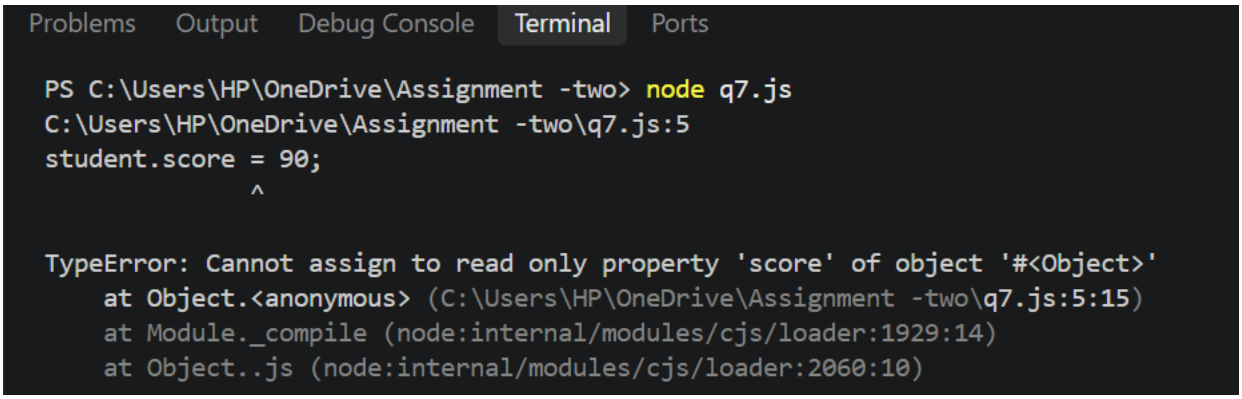
```
  score: 50
};

Object.freeze(student);

// Try to change an existing property
student.score = 90;

console.log(student);
```

Output:



Because we used "use strict", JavaScript throws a TypeError when we attempt to change the frozen object's property. The error occurs before console.log(student) executes, so the object is not printed by that line. The object's score remains 50.

Why does the error fail silently in normal mode?

In JavaScript's normal mode, also called sloppy mode, attempting to modify a frozen object's property does not throw an error. Instead, the assignment is ignored.

For example:

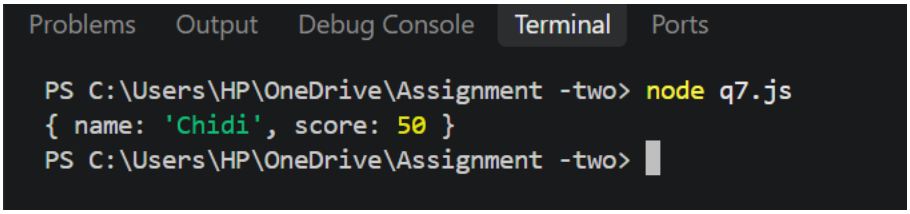
```
const student = {
  name: "Chidi",
  score: 50
};
```

```
Object.freeze(student);

student.score = 90;

console.log(student);
```

Output:



The assignment fails silently because JavaScript's normal mode does not throw an error when an assignment to a non-writable property fails. The object's score remains unchanged. Strict mode makes this mistake easier to detect by throwing an error instead.

One important limitation of Object.freeze()

Object.freeze() is shallow, meaning it freezes only the object's immediate properties. If an object contains another object, that nested object can still be modified.

For example:

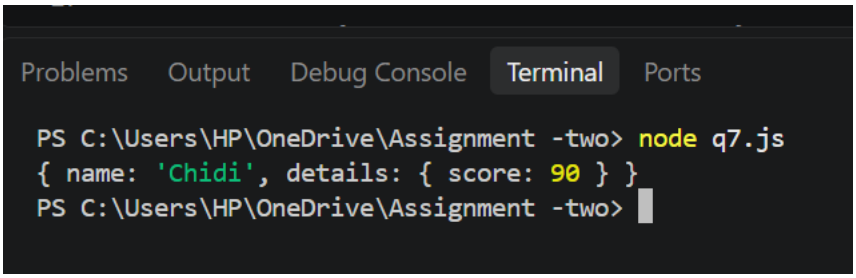
```
const student = {
  name: "Chidi",
  details: {
    score: 50
  }
};

Object.freeze(student);
```

```
student.details.score = 90;
```

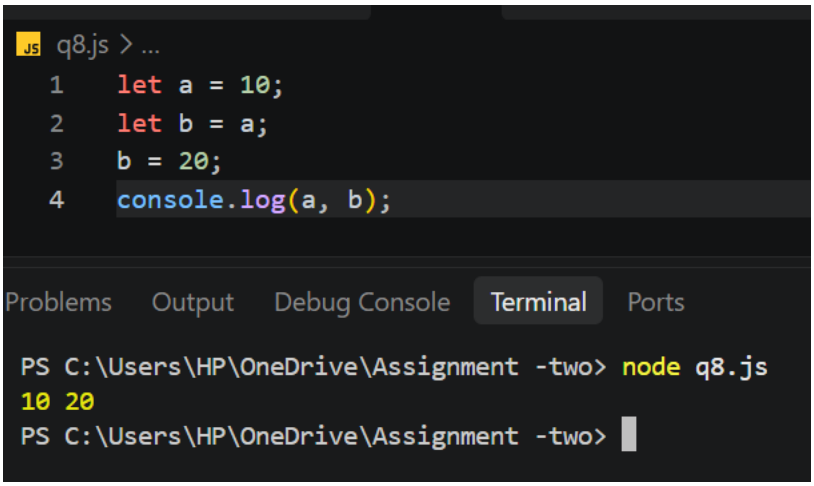
```
console.log(student.details.score);
```

Output:

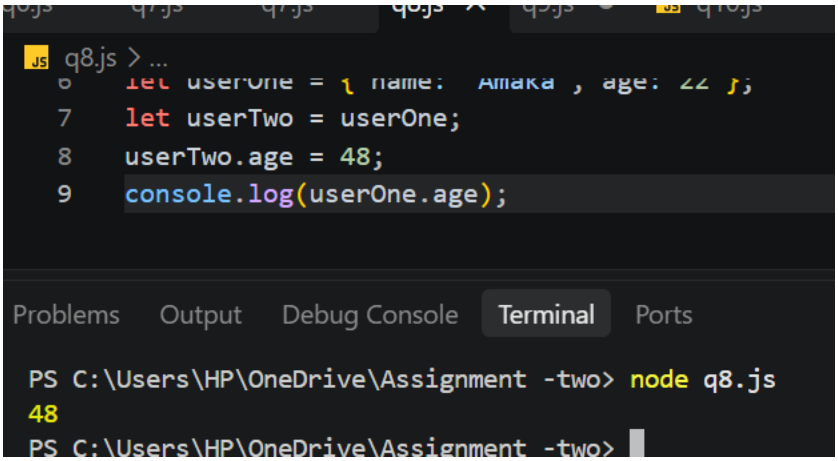


This works because the nested details object has not been frozen. To prevent changes to nested objects as well, you would need to freeze them separately or implement a deep-freezing function.

QUESTION 8



Example 1: Primitive values, when we assign a to b, JavaScript copies the value 10 into both variables hold their own independent values. when we change b to 20, it does not affect a, which remains 10.



Example 2: Objects (Non-Primitive Values) when we assign userOne to userTwo, JavaScript copies the reference to the object, not an independent copy of the object itself. Both variables refer to the same object in memory. Therefore, changing userTwo.age also changes the age we see through userOne.

8a

Why do the two blocks behave differently?

The difference comes down to copying a value versus copying a reference.

1. Primitive values: Copying a value

Primitive types include numbers, strings, booleans, undefined, null, bigint, and symbols.

For example:

```
let a = 10;

let b = a;


b = 20;


console.log(a); // 10

console.log(b); // 20
```

Here, b receives a copy of the value stored in a. Changing b does not affect a.

Think of it as writing the number 10 on two separate pieces of paper. Changing the number on one paper does not change the other.

2. Non-primitive values: Copying a reference

Objects and arrays are non-primitive values.

Consider this example:

```
let userOne = { name: "Amaka", age: 22 };

let userTwo = userOne;


userTwo.age = 40;


console.log(userOne.age); // 40

console.log(userTwo.age); // 40
```

Both variables refer to the same object. When we change a property through userTwo, we also see that change when accessing the object through userOne.

Think of it as two people having directions to the same house. If someone changes something inside the house, both people are still referring to that same house.

8B

Fix the second block using the spread operator

We can use the spread operator (...) to make a shallow copy of the object.

Example 1: Copying a simple object

```
let userOne = { name: "Amaka", age: 22 };

let userTwo = { ...userOne };

userTwo.age = 40;


console.log(userOne.age); // 22

console.log(userTwo.age); // 40
```

The expression { ...userOne } creates a new object containing the same properties and values as userOne .Now, userOne and userTwo refer to two separate objects. Changing the age in userTwo does not affect userOne.This works because the properties in this example contain primitive values.

Example 2: What happens when the object contains another object?

Let's add an address property to the object.

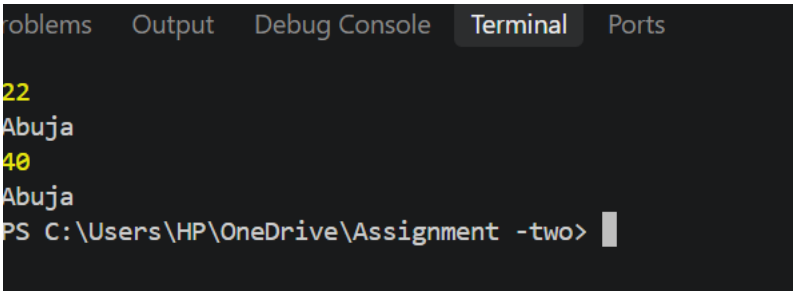
```
let userOne = {
  name: "Amaka",
  age: 22,
  address: {
    city: "Lagos",
    country: "Nigeria"
  }
};

let userTwo = { ...userOne };

userTwo.age = 40;
userTwo.address.city = "Abuja";

console.log(userOne.age);
console.log(userOne.address.city);

console.log(userTwo.age);
console.log(userTwo.address.city);
```



Why did this happen?

The spread operator creates a shallow copy, not a deep copy.

This means: The top level properties are copied into a new object. Primitive values, such as age, are copied independently. The nested address object is not copied independently. Both users' objects refer to the same nested address object. Therefore, changing userTwo.age does not affect userOne.age, but changing userTwo.address.city also changes userOne.address.city.

How can we fix the nested object problem?

For this example, we can use the spread operator twice to copy both the outer object and the nested address object.

```
let userOne = {
  name: "Amaka",
  age: 22,
  address: {
    city: "Lagos",
    country: "Nigeria"
  }
};


let userTwo = {
  ...userOne,
  address: { ...userOne.address }
};
```

```
userTwo.age = 40;
userTwo.address.city = "Abuja";

console.log(userOne);
console.log(userTwo);
Output:
{
  name: "Amaka",
  age: 22,
  address: {
    city: "Lagos",
    country: "Nigeria"
  }
}

{
  name: "Amaka",
  age: 40,
  address: {
    city: "Abuja",
    country: "Nigeria"
  }
}
```

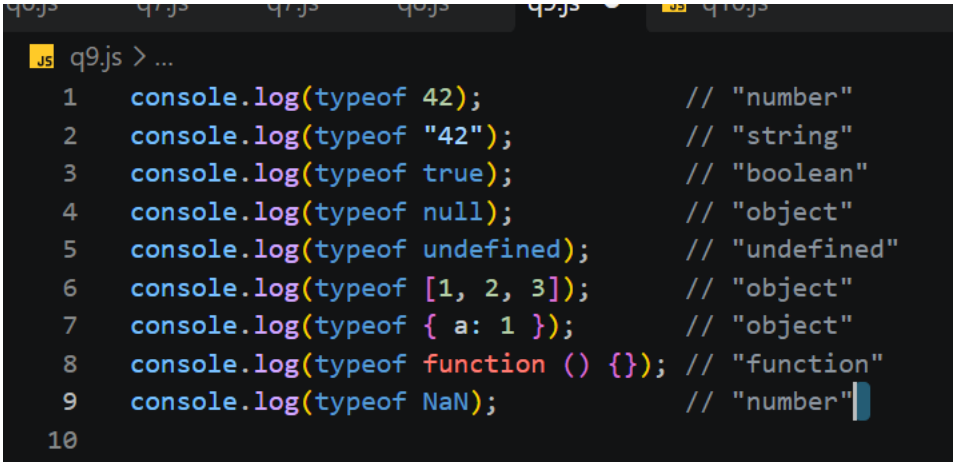
Now, changing userTwo.address.city does not affect userOne.address.city, because the nested address objects are separate. This solution works for this particular structure. If an object contains several levels of nested objects, you must copy each nested level you want

to modify independently, or use an appropriate deep-copying approach.

QUESTION 9

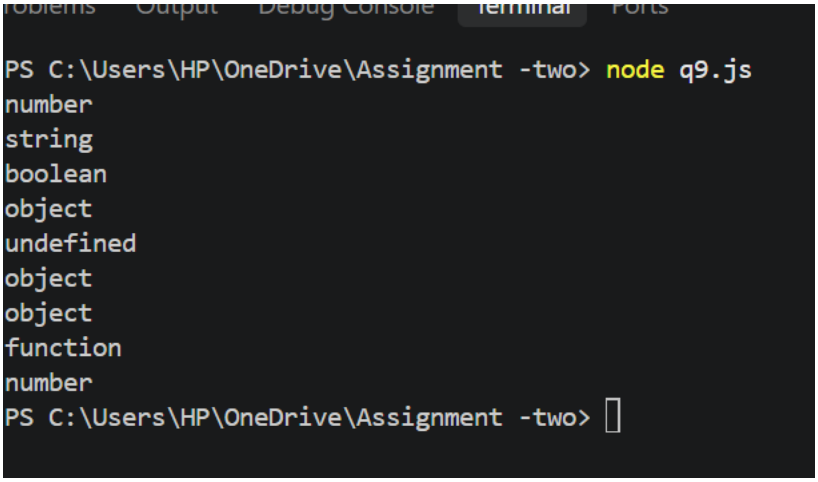
9A

Predict the output of typeof



```
q9.js > ...
1   console.log(typeof 42);             // "number"
2   console.log(typeof "42");           // "string"
3   console.log(typeof true);           // "boolean"
4   console.log(typeof null);           // "object"
5   console.log(typeof undefined);      // "undefined"
6   console.log(typeof [1, 2, 3]);      // "object"
7   console.log(typeof { a: 1 });       // "object"
8   console.log(typeof function () {}); // "function"
9   console.log(typeof NaN);            // "number"
10
```

Output



```
PS C:\Users\HP\OneDrive\Assignment -two> node q9.js
number
string
boolean
object
undefined
object
object
function
number
PS C:\Users\HP\OneDrive\Assignment -two>
```

9b

Two surprising results

1. **typeof null returns "object"**

This is a historical JavaScript bug. It can confuse developers checking whether a value exists.

2**. typeof [1, 2, 3] returns "object"**

Arrays are objects in JavaScript, but developers often need to identify them separately. For example, a backend might receive an array when it expects an object. Checking only typeof could allow the wrong data through.

To check for these cases:

console.log(value === null); // Checks for null

console.log(Array.isArray(value)); // Checks for arrays

9c

Write a function called getType(value)

```
function getType(value) {
    if (value === null)return 'null';
    if (Array.isArray(value)) return 'array';
    return typeof value;
}
console.log(getType(42));
"number"
console.log(getType("42"));
"string"
console.log(getType(true));
"boolean"
console.log(getType(null));
"null"
console.log(getType(undefined));
"undefined"
console.log(getType([1, 2, 3]));
"array"
console.log(getType({ a: 1 }));
"object"
console.log(getType(function () {}));
"function"
console.log(getType(NaN));
"number"
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\HP\OneDrive\Assignment -two> node q9.js
number
string
boolean
null
undefined
array
object
function
number
PS C:\Users\HP\OneDrive\Assignment -two>
```

QUESTION 10

10a

Convert Seconds into Days, Hours, Minutes, and Seconds

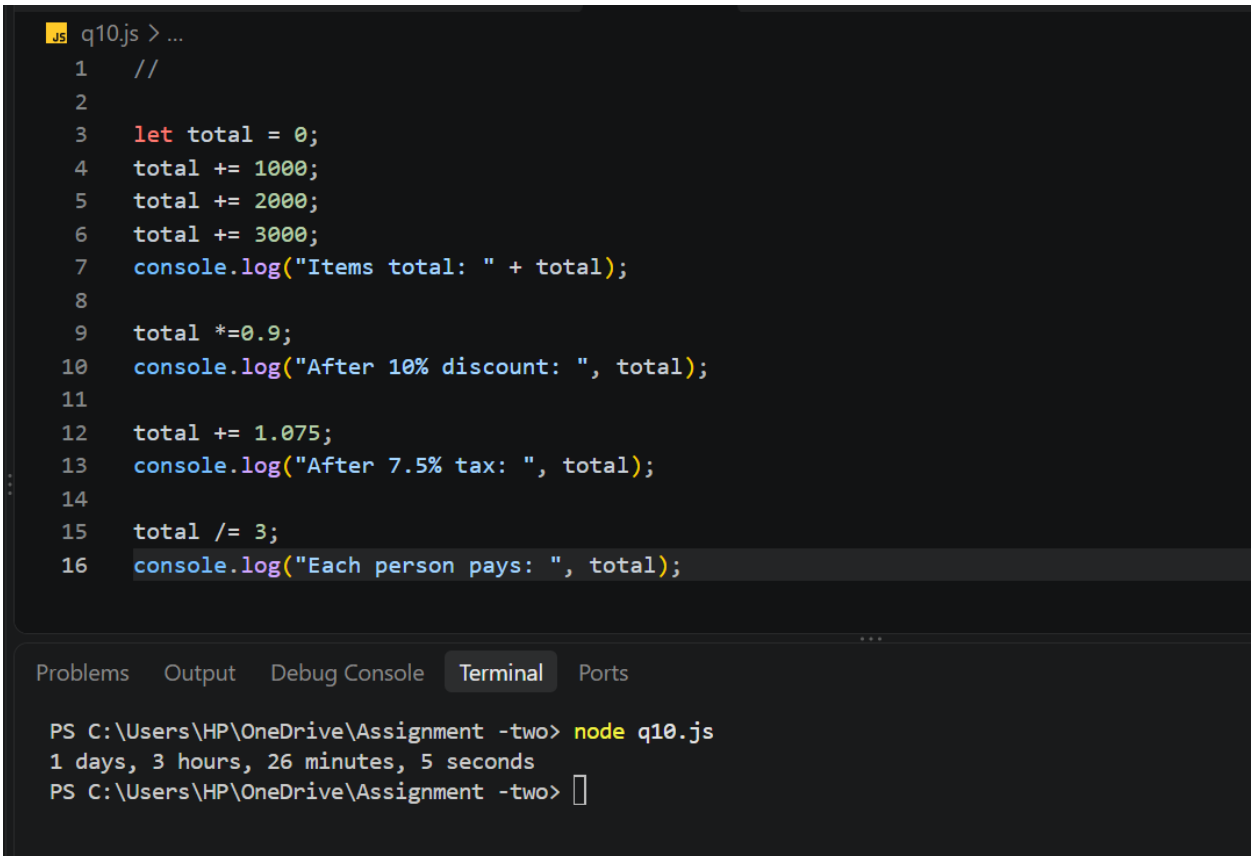
```
q10.js > ...
1   let seconds = 98765;
2
3   let days = Math.floor(seconds / 86400);
4   seconds %= 86400;
5
6   let hours = Math.floor(seconds / 3600);
7   seconds %= 3600;
8
9   let minutes = Math.floor(seconds / 60);
10  seconds %= 60;
11  console.log(`${days} days, ${hours} hours, ${minutes} minutes, ${seconds} seconds`);
```

```
Problems  Output  Debug Console  Terminal  Ports  powershell

PS C:\Users\HP\OneDrive\Assignment -two> node q10.js
1 days, 3 hours, 26 minutes, 5 seconds
PS C:\Users\HP\OneDrive\Assignment -two>
```

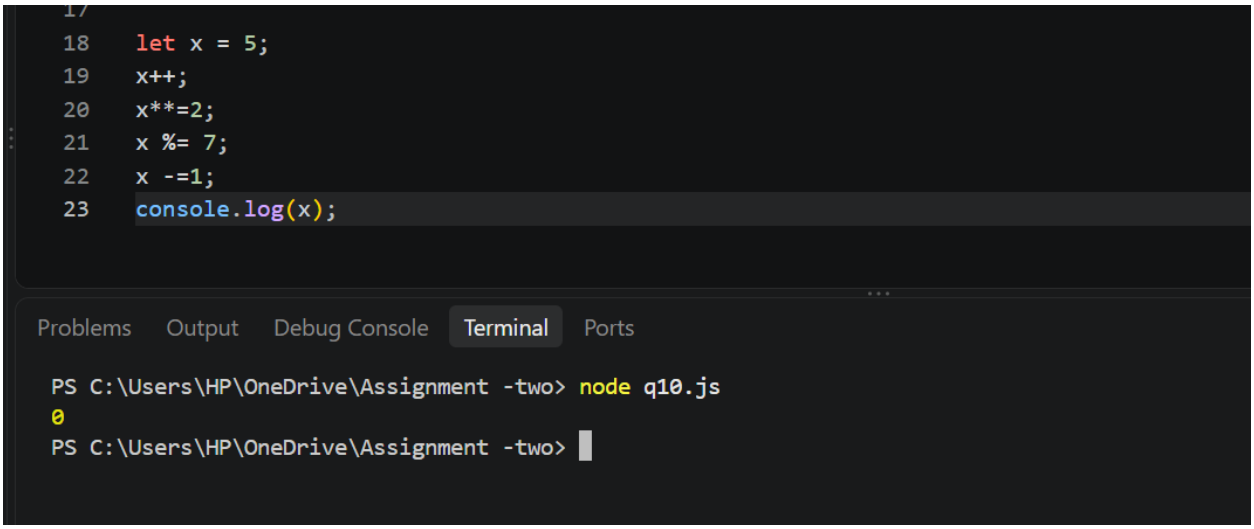
10b

Shop Total Calculator



10c

Predict the Value of x



x++ increases x by 1.

x **= 2 squares x.

x %= 7 gives the remainder after dividing by 7.

x -= 1 subtracts 1.